COMP 424 - Homework 2

Antoine Boubez

Question 1: Constraint satisfaction [50]

1(a) Formulate this problem as a CSP. List the variables, their domains, and the constraints. Draw the constraint graph.

variables :
one per empty cell Xij -> X12, X14, X21, X23, X32, X33, X41, X42, X43, X44

domains: every variable Xij's value is element of D = {1,2,3,4}

constraints:
Givens: X11 = 2, X13 = 4, X22 = 1, X24 = 2, X31 = 1, X34 = 3

Rows: Xij ≠ Xik for every row i, and every j ≠ k
Columns: Xij ≠ Xkj for every column j, and every i ≠ k

Boxes: Xij ≠ Xkl for every (i, j) ≠ (k, l) in the same 2x2 box:
{X11, X12, X21, X22}, {X13, X14, X23, X24},
{X31, X32, X41, X42}, {X33, X34, X43, X44}
constraint graph :
image is at the end of the document

1(b) Show the first ten steps of backtracking search on this problem, where you order the variables in increasing order first by row, then by column, and the values from lowest to highest. Recall that backtracking search uses a depth-first strategy to expand the search tree.
(0) starting grid (c= colum, r = row, b=box)

```
2 . | 4 .
. 1 | . 2
----+----
1 . | . 3
. . | . .
```

(1) X12 =3 (c2 has 1, r1 has 2, -> 3)

```
2 3 | 4 .
. 1 | . 2
----+----
1 . | . 3
. . | . .
```

(2) X14 = 1 (-> 1)

```
2 3 | 4 1
. 1 | . 2
----+----
1 . | . 3
. . | . .
(3) X21 = 4 (r2 has 1, r2 has 2, b has 3, -> 4)
2 3 | 4 1
4 1 | . 2
----+----
1 . | . 3
. . | . .
(4) X23 = 3 (r2 has 1, r2 has 2, -> 3)
2 3 | 4 1
4 1 | 3 2
----+----
1 . | . 3
. . | . .
(5) X32 =2 (r3 has 1, -> 2)
2 3 | 4 1
4 1 | 3 2
----+----
1 2 | . 3
. . | . .
(6) X33 -> no solution (r3 has 1, r3 has 2, r3 has 3, c3 has 4 -> backtrack to X32)
2 3 | 4 1
4 1 | 3 2
----+----
1 2 | . 3
. . | . .
(7) X32 = 4 (continues from step 5, r3 has 3, -> 4)
2 3 | 4 1
4 1 | 3 2
----+----
1 4 | . 3
. . | . .
(8) X33 = 2 ( r3 has 1, ->2)
2 3 | 4 1
4 1 | 3 2
----+----
1 4 | 2 3
. . | . .
(9) X41  = 3 (c1 has 1, c1 has 2, ->3)
2 3 | 4 1
4 1 | 3 2
----+----
1 4 | 2 3
3 . | . .
(10) X42 =2 (c2 has 1, ->2)
2 3 | 4 1
4 1 | 3 2
```

```
----+----
1 4 | 2 3
3 2 | . .
```

1(c) Show the first ten steps of backtracking search on this problem with one-step forward checking, where you order the variables in increasing order first by row, then by column, and the values from lowest to highest.

```
(0) starting grid
domains after pruning with the givens:
X12{3} X14{1} X21{3,4} X23{3} X32{2,4} X33{2} X41{3,4} X42{2,3,4} X43{1,2} X44{1,4}
2 . | 4 .
. 1 | . 2
----+----
1 . | . 3
. . | . .
(1) X12 =3, prunes :  X21 -> {4}, X42 -> {2,4}
2 3 | 4 .
. 1 | . 2
----+----
1 . | . 3
. . | . .
(2) X14 = 1 , prunes : X44 - >{4}
2 3 | 4 1
. 1 | . 2
----+----
1 . | . 3
. . | . .
(3) X21 = 4 , prunes :  X41-> {3}
2 3 | 4 1
4 1 | . 2
----+----
1 . | . 3
. . | . .
(4) X23 = 3 , prunes : null
2 3 | 4 1
4 1 | 3 2
----+----
1 . | . 3
. . | . .
(5) X32 =2 , prunes : X33 ->{}
impossible -> 2 is rejected
2 3 | 4 1
4 1 | 3 2
----+----
1 2 | . 3
. . | . .
```

```
(6) X32 = 4, prunes : X42 -> {2}
2 3 | 4 1
4 1 | 3 2
----+----
1 4 | . 3
. . | . .
(7) X33 =2, prunes : X43 -> {1}
2 3 | 4 1
4 1 | 3 2
----+----
1 4 | 2 3
. . | . .
(8) X41 =3, prunes : null
2 3 | 4 1
4 1 | 3 2
----+----
1 4 | 2 3
3 . | . .
(9) X42 = 2 , prunes : null
2 3 | 4 1
4 1 | 3 2
----+----
1 4 | 2 3
3 2 | . .
(10)X43 = 1, prunes : null
2 3 | 4 1
4 1 | 3 2
----+----
1 4 | 2 3
3 2 | 1 .
```

1(d) Write a simple program to implement a simulated annealing local search strategy to solve this problem. Submit in writing your problem formulation and document the algorithm design decisions that you make. Run your algorithm with at least three different sets of design choices (e.g., annealing schedule, initial temperature). Show the first five steps of the algorithm on one run of the program for one setting, including the game state and the number of constraint violations in each step. Run the program 100 times with a maximum of 10000 steps per run. For each of the three settings, report the number of runs in which the program finds a solution, and make note of any other interesting points you observe.

Code: q1d_simulated_annealing.py
Problem formulation
A state is a 4x4 grid where the original 6 prefilled cells never change and where every 2x2 box contains
each digit once. The initial state is built by filling the remaining cells randomly

from the available numbers.
A neighbour is obtained by choosing a box and swapping 2 of the non-given cells. The cost of a state is the number
of repeated digits across all rows and columns. This is the equivalent of 4 minus the number of
distinct digits in each row and column, added up over all 4 rows and 4 columns, so a correct row or column contributes 0
and each repeated digit adds 1. The boxes are always correct by construction.

Design decisions
The neighbours are chosen by swapping inside a box so that the box constraint is always respected, and the search
only has to arrange the rows/columns. This also makes the search space small: the 4 boxes have 2, 2, 3 and 3 free cells,
so there are only 2 x 2 x 6 x 6 =144 possible states. On each step, a neighbour with a lower cost is always accepted.
A neighbour with an equal or higher cost is accepted with probability exp(-delta /T), where delta is the new cost minus the
current cost. After every step the temperature is lowered by multiplying it by alpha, so T = T x alpha. A run stops when the cost
reaches 0, or after 10000 steps, in which case it returns the best solution we found yet.

Settings
A: T0 = 1.0, a = 0.999
B: T0 = 5.0, a = 0.99
 C: T0 = 0.1, a = 0.9

First five steps, setting A
start, violations = 8
```
  2 4 | 4 3
  3 1 | 1 2
  ----+----
  1 2 | 1 3
  4 3 | 4 2
```

step 1 T = 1.0 delta = -2 accepted violations = 6
```
  2 4 | 4 3
  3 1 | 1 2
  ----+----
  1 2 | 1 3
  4 3 | 2 4
```

step 2 T = 0.999 delta = -2 accepted violations = 4
```
  2 4 | 4 1
  3 1 | 3 2
  ----+----
  1 2 | 1 3
  4 3 | 2 4
```

```
step 3 T = 0.998 delta = 2 rejected violations = 4
  2 4 | 4 1
  3 1 | 3 2
  ----+----
  1 2 | 1 3
  4 3 | 2 4

step 4 T = 0.997 delta = 0 accepted violations = 4
  2 3 | 4 1
  4 1 | 3 2
  ----+----
  1 2 | 1 3
  4 3 | 2 4

step 5 T = 0.996 delta = 0 accepted violations = 4
  2 3 | 4 1
  4 1 | 3 2
  ----+----
  1 2 | 4 3
  4 3 | 2 1

Results over 100 runs, max 10000 steps
A: T0=1.0, alpha=0.999
  solved: 100 / 100
avg steps: 42.3
max steps: 187
B: T0=5.0, alpha=0.99
  solved: 100 / 100
avg steps: 71.4
max steps: 227
C: T0=0.1, alpha=0.9
  solved: 100 / 100
avg steps: 38.1
max steps: 145

Solution found:
  2 3 | 4 1
  4 1 | 3 2
  ----+----
  1 4 | 2 3
  3 2 | 1 4

Observations
The puzzle was solved 100/100 times for all 3 different settings. No run came close
to the max step limit. (max 227 steps)
This makes sense since there are only 144 possible states.  B was the slowest (71.4
avg) because it accepted worse moves at a higher temperature.
C was the fastest (38.1 avg) it behaved similar to hill climbing because of the
very low temp.  This worked well since
```

the search space is small. Moves with delta 0 are always accepted which lets it move across plateau.
All runs had the same solution. (the only one)

Question 2: Search and Game Playing [30]

2(a) Apply the Minimax algorithm to the above state, by preferring horizontal moves to vertical moves, and expanding in increasing order first by row, then by column. For convenience, you can represent the moves as (start row, start column, end row, end column), e.g. drawing a vertical line in the bottom right corner can be represented as the move (3, 3, 2, 3) or equivalently (2, 3, 3, 3). Draw the corresponding search tree.
[Note: the preference for horizontal moves affects how you apply the Minimax algorithm, but not how the players play the game.]

TL/TR/BR /BL = top-left / top-right / bottom-right / bottom-right

```
MAX root = -1
|
+--(1,1,1,2)-- MIN = -1
|   |
|   +--(2,2,2,3)-- MAX = +1
|   |   +--(1,2,2,2) [MAX: TL,TR]-- MIN = +1 --(2,3,3,3) [MIN: BR]-- +1
|   |   +--(2,3,3,3) [MAX: BR]-- MIN = -1 --(1,2,2,2) [MIN: TL,TR]-- -1
|   |
|   +--(1,2,2,2) [MIN: TL]-- MAX = -1
|   |   +--(2,2,2,3) [MAX: TR]-- MIN = -1 --(2,3,3,3) [MIN: BR]-- -1
|   |   +--(2,3,3,3)-- MIN = -3 --(2,2,2,3) [MIN: TR,BR]-- -3
|   |
|   +--(2,3,3,3)-- MAX = -1
|       +--(2,2,2,3) [MAX: BR]-- MIN = -1 --(1,2,2,2) [MIN: TL,TR]-- -1
|       +--(1,2,2,2) [MAX: TL]-- MIN = -1 --(2,2,2,3) [MIN: TR,BR]-- -1
|
|
+--(2,2,2,3)-- MIN = -1
|   |
|   +--(1,1,1,2)-- MAX = +1
|   |   +--(1,2,2,2) [MAX: TL,TR]-- MIN = +1 --(2,3,3,3) [MIN: BR]-- +1
|   |   +--(2,3,3,3) [MAX: BR]-- MIN = -1 --(1,2,2,2) [MIN: TL,TR]-- -1
|   |
|   +--(1,2,2,2) [MIN: TR]-- MAX = -1
|   |   +--(1,1,1,2) [MAX: TL]-- MIN = -1 --(2,3,3,3) [MIN: BR]-- -1
|   |   +--(2,3,3,3) [MAX: BR]-- MIN = -1 --(1,1,1,2) [MIN: TL]-- -1
|   |
|   +--(2,3,3,3) [MIN: BR]-- MAX = -1
|       +--(1,1,1,2)-- MIN = -3 --(1,2,2,2) [MIN: TL,TR]-- -3
|       +--(1,2,2,2) [MAX: TR]-- MIN = -1 --(1,1,1,2) [MIN: TL]-- -1
|
```

```
|
+--(1,2,2,2)-- MIN = -1
|   |
|   +--(1,1,1,2) [MIN: TL]-- MAX = -1
|   |   +--(2,2,2,3) [MAX: TR]-- MIN = -1 --(2,3,3,3) [MIN: BR]-- -1
|   |   +--(2,3,3,3)-- MIN = -3 --(2,2,2,3) [MIN: TR,BR]-- -3
|   |
|   +--(2,2,2,3) [MIN: TR]-- MAX = -1
|   |   +--(1,1,1,2) [MAX: TL]-- MIN = -1 --(2,3,3,3) [MIN: BR]-- -1
|   |   +--(2,3,3,3) [MAX: BR]-- MIN = -1 --(1,1,1,2) [MIN: TL]-- -1
|   |
|   +--(2,3,3,3)-- MAX = +1
|       +--(1,1,1,2) [MAX: TL]-- MIN = -1 --(2,2,2,3) [MIN: TR,BR]-- -1
|       +--(2,2,2,3) [MAX: TR,BR]-- MIN = +1 --(1,1,1,2) [MIN: TL]-- +1
|
|
+--(2,3,3,3)-- MIN = -1
    |
    +--(1,1,1,2)-- MAX = -1
    |   +--(2,2,2,3) [MAX: BR]-- MIN = -1 --(1,2,2,2) [MIN: TL,TR]-- -1
    |   +--(1,2,2,2) [MAX: TL]-- MIN = -1 --(2,2,2,3) [MIN: TR,BR]-- -1
    |
    +--(2,2,2,3) [MIN: BR]-- MAX = -1
    |   +--(1,1,1,2)-- MIN = -3 --(1,2,2,2) [MIN: TL,TR]-- -3
    |   +--(1,2,2,2) [MAX: TR]-- MIN = -1 --(1,1,1,2) [MIN: TL]-- -1
    |
    +--(1,2,2,2)-- MAX = +1
        +--(1,1,1,2) [MAX: TL]-- MIN = -1 --(2,2,2,3) [MIN: TR,BR]-- -1
        +--(2,2,2,3) [MAX: TR,BR]-- MIN = +1 --(1,1,1,2) [MIN: TL]-- +1
```

2(b) Apply the alpha-beta pruning method using the same order of node expansion and show the alpha-beta values for all nodes. Is there any advantage to using alpha-beta pruning?

```
MAX root [-1, +inf] ret -1
|
+--(1,1,1,2)-- MIN [-inf, -1] ret -1
|   |
|   +--(2,2,2,3)-- MAX [+1, +inf] ret +1
|   |   +--(1,2,2,2) [MAX: TL,TR]-- MIN [-inf, +1] ret +1 --(2,3,3,3) [MIN: BR]--
+1
|   |   +--(2,3,3,3) [MAX: BR]-- MIN [+1, -1] ret +1 --(1,2,2,2) [MIN: TL,TR]-- -1
|   |
|   +--(1,2,2,2) [MIN: TL]-- MAX [-1, +1] ret -1
|   |   +--(2,2,2,3) [MAX: TR]-- MIN [-inf, -1] ret -1 --(2,3,3,3) [MIN: BR]-- -1
|   |   +--(2,3,3,3)-- MIN [-1, -3] ret -1 --(2,2,2,3) [MIN: TR,BR]-- -3
|   |
```

```
|   +--(2,3,3,3)-- MAX [-1, -1] ret -1
|       +--(2,2,2,3) [MAX: BR]-- MIN [-inf, -1] ret -1 --(1,2,2,2) [MIN: TL,TR]--
-1
|       +--(1,2,2,2) [MAX: TL]-- PRUNED
|
|
+--(2,2,2,3)-- MIN [-1, -1] ret -1
|   |
|   +--(1,1,1,2)-- MAX [+1, +inf] ret +1
|   |   +--(1,2,2,2) [MAX: TL,TR]-- MIN [-1, +1] ret +1 --(2,3,3,3) [MIN: BR]-- +1
|   |   +--(2,3,3,3) [MAX: BR]-- MIN [+1, -1] ret +1 --(1,2,2,2) [MIN: TL,TR]-- -1
|   |
|   +--(1,2,2,2) [MIN: TR]-- MAX [-1, +1] ret -1
|   |   +--(1,1,1,2) [MAX: TL]-- MIN [-1, -1] ret -1 --(2,3,3,3) [MIN: BR]-- -1
|   |   +--(2,3,3,3) [MAX: BR]-- MIN [-1, -1] ret -1 --(1,1,1,2) [MIN: TL]-- -1
|   |
|   +--(2,3,3,3) [MIN: BR]-- PRUNED
|
|
+--(1,2,2,2)-- MIN [-1, -1] ret -1
|   |
|   +--(1,1,1,2) [MIN: TL]-- MAX [-1, +inf] ret -1
|   |   +--(2,2,2,3) [MAX: TR]-- MIN [-1, -1] ret -1 --(2,3,3,3) [MIN: BR]-- -1
|   |   +--(2,3,3,3)-- MIN [-1, -3] ret -1 --(2,2,2,3) [MIN: TR,BR]-- -3
|   |
|   +--(2,2,2,3) [MIN: TR]-- PRUNED
|   +--(2,3,3,3)-- PRUNED
|
|
+--(2,3,3,3)-- MIN [-1, -1] ret -1
    |
    +--(1,1,1,2)-- MAX [-1, +inf] ret -1
    |   +--(2,2,2,3) [MAX: BR]-- MIN [-1, -1] ret -1 --(1,2,2,2) [MIN: TL,TR]-- -1
    |   +--(1,2,2,2) [MAX: TL]-- MIN [-1, -1] ret -1 --(2,2,2,3) [MIN: TR,BR]-- -1
    |
    +--(2,2,2,3) [MIN: BR]-- PRUNED
    +--(1,2,2,2)-- PRUNED
```

Question 3: Search Under Uncertainty [20]

3(a) Play a game of Connections at
https://www.nytimes.com/games/connections to familiarize yourself
with the rules. (No marks)

3(b) Formulate the Connections game for a given day as a search problem,
formally showing all parts of the search problem as shown in the
lectures. Is this an observable, partially observable, or

non-observable problem? Deterministic or not?

P = the way to split of the 16 words into groups of 4. (the answer)
number of splits : 16!/ ((4!)^4 x 4!) = 2627625

states:
-the set of splits that could still be P, given what we know.
-the groups found
-allowed number of mistakes left

initial state:
-all 2627625 splits
-no groups found
-4 errors left

actions:
guess 4 of the remaining words (have not been guessed before)

percepts:
after each try, determine if its correct/wrong/one away

perception function:
if all 4 guessed words are in the same group -> correct
if 3 of the 4 guessed words are in teh same group -> one away
otherwise -> wrong

transitions
correct -> remove those 4 words from the pool and the group counts as found
one away or wrong -> counts as a mistake and you have one less mistake remaining
in all cases -> remove every split that would not have given this percept (it can't be P)

goal :
find all 4 groups (P) before running out of mistakes.

path cost:
number of mistakes made

It is partially observable since we can see the words, the group found and the number of mistakes left but we cannot see any goups or P.
Each percept can only tell us an incomplete part of the answer, so we have to keep track of all the splits that could still be P.

It is deterministic since P doesn't change during the game, so the same guess in the same state always gets the same percept.
The uncertainty comes from it being partially observable, not from the actions having undetermined outcomes.

3(c) Describe how you could attempt to solve the Connections search problem using an AND-OR search tree. What would the AND and OR nodes

represent? How many branches are at each AND and OR node? Is this a good idea in practice? Justify your answer.

root: start of the game (all splits possible, no groups found, 4 mistakes left)

-alternate between our guess (OR node) and the game's percept (AND node)
-repeat until the game is won or lost

-solution = a plan that says which guess to make after every possible percept, and wins in every branch

what they represent :
OR nodes: our choice of which 4 words to guess (we choose the action)
AND nodes: the percept the game gives after a guess -> correct/one away/wrong (the game chooses)

number of branches:
OR nodes: one per possible guess -> at most C(16,4) = 1820 at the start, then at most 495, then at most 70,
then 1 (fewer if some guesses were already tried)
AND nodes: one per percept that is still possible -> at most 3 (only 1 when 4 words are left, since the guess
has to be correct)

is it a good idea in practice ?
No
-the tree is too big: the first OR node already has 1820 branches, each guess has up to 3 percepts, and we have
to keep track of 2627625 possible splits
-a winning plan doesn't exist: a game has at most 7 guesses (3 mistakes + 4 correct) and each guess has at most
3 percepts -> a plan can only end in at most 3^7 = 2187 different ways. Each way it wins only works for one P,
so a plan can win for at most 2187 of the 2627625 splits -> the AND-OR search would return failure
-the tree doesnt use the meaning of the words and only looks at which splits are still possible. Doing it this way keeps so many options
and makes the tree too big.